Energy Harvesting Aware Path Planning for Ambiently-Powered Multi-Robot Systems

Mohmmadsadegh Mokhtari[1], Bram Vanderborght[2], and Jeroen Famaey[1]

[1]University of Antwerp – imec, Antwerp, Belgium
[2]Vrije Universiteit Brussel – imec, Brussels, Belgium

Corresponding author: Mohmmadsadegh Mokhtari, 2000 Antwerp, Belgium
Mohmmadsadegh.Mokhtari@uantwerpen.be

November 10, 2025

Abstract

Autonomous multi-robot systems are increasingly deployed in energy-variable environments where sustained operation depends on efficient energy harvesting. Traditional path-planning methods overlook ambient energy variability and inter-robot coordination, reducing overall efficiency. This paper introduces Hierarchically Conditioned Multi-Agent Reinforcement Learning with Meta-Differential Evolution (H-CMARL-DE) for energy harvesting-aware multi-robot path planning. The method centralizes only meta-level parameters priority order, shaping weights, and subgoal settings, while policy learning and execution remain decentralized, ensuring scalability and safety. Implemented in ROS-Gazebo, the system operates in a closed-loop observe-plan-act cycle, enabling robots to coordinate via hierarchical time–space reservations while adapting to dynamic obstacles and energy fields. Simulations demonstrate up to 240% improvement in energy-harvesting efficiency with minimal increase in path length compared to non-energy-harvesting-aware approaches, confirming H-CMARL-DE's robustness and adaptability for long-term cooperative operation in resource-constrained environments.

Index terms: Energy Harvesting awareness, Path Planning, Ambiently Powered Multi Robot Systems, Deep Reinforcement Learning.

1 Introduction

Multi-robot systems represent a key advancement in autonomous robotics, enabling teams of robots to cooperatively perform complex missions such as environmental monitoring, warehouse logistics, and disaster response. These systems are increasingly deployed in energy-constrained environments, where continuous access to charging infrastructure is impractical. To achieve long-term autonomy, many robots now integrate ambient energy harvesting (EH) technologies, such as solar, thermal, or wireless inductive charging, allowing them to sustain operations without frequent manual recharging [1].

However, ambient energy availability is inherently variable in both space and time. Factors such as illumination, wireless signal strength, and environmental occlusion lead to dynamic fluctuations in energy supply. Consequently, planning energy-efficient and energy-harvesting-aware trajectories becomes a challenging problem, as robots must simultaneously minimize travel cost and maximize energy intake while avoiding collisions and maintaining task efficiency. A naive shortest-path planner may neglect EH opportunities, while an EH-maximizing policy may overextend travel time. Thus, an effective solution must balance path optimality, energy harvesting, and coordination among multiple agents.

Traditional path-planning algorithms, such as A^* and its energy-augmented variants, perform well in static environments but lack adaptability to dynamic conditions and inter-robot dependencies. Reinforcement

Learning (RL) approaches, on the other hand, can learn adaptive behaviors through trial-and-error interactions with the environment. Yet, standard multi-agent RL frameworks—particularly those following centralized training with decentralized execution (CTDE)—often suffer from scalability and communication overhead when the number of agents increases.

To address these limitations, this paper introduces H-CMARL-DE, a Hierarchically Conditioned Multi-Agent Reinforcement Learning framework enhanced with meta-level Differential Evolution (DE) optimization. Unlike fully centralized or decentralized schemes, H-CMARL-DE centralizes only high-level meta-variables, including priority order, shaping weights, and subgoal configurations, while keeping policy learning and execution entirely decentralized. This design enables scalable coordination with minimal communication and computational cost.

The framework employs a hierarchical time–space reservation mechanism to manage inter-robot conflicts and improve cooperation efficiency. Differential Evolution is used at the meta-level to optimize global parameters such as reward shaping and coordination weights, enabling adaptive tuning across training episodes. Implemented in ROS–Gazebo, the system operates within a closed-loop observe–plan–act cycle, allowing robots to sense ambient energy distributions, adjust their paths dynamically, and coordinate their movements efficiently.

Experimental results demonstrate that H-CMARL-DE improves energy-harvesting efficiency by up to 240% compared to baseline planners, with only marginal increases in path length and mission completion time. This confirms the framework's potential for scalable, energy-sustainable coordination of autonomous robotic swarms operating in resource-variable environments.

The rest of this paper is organized as follows: Section 2 reviews related work in energy-aware and multi-agent RL-based path planning. Section 3 formulates the problem setup. Section 4 illustrates the baselines and proposed methods, Section 5 reports simulation results, Section 6 presents a case study in ROS and Gazebo platform, and Section 7 concludes the paper.

2 Related Work

RL and DRL have emerged as a powerful approach to addressing challenges in robot navigation, enabling robots to learn optimal strategies through interaction with their environment. Table 1 shows the detailed comparison of the RL path planning method [2] to other conventional methods like Dijkstra [3], Dynamic Rapidly-Exploring Random Tree (DRRT) [4], and A^* Algorithm [5]. Based on the table, it can be concluded that RL is best suited for EH-aware path planning due to its ability to handle dynamic, stochastic environments and incorporate multiple objectives. Unlike A^*, DRRT, and Dijkstra, RL adapts to changes naturally and generalizes from experience. It is robust to noise and uncertainties, making it ideal for real-time applications. While RL has high training costs, it offers fast runtime once trained. In contrast, other methods struggle with scalability, adaptability, and handling multiple objectives.

2.1 Recent Advances in Path Planning

Recent advances in multi-agent and robotic reinforcement learning have explored various forms of centralized training, decentralized execution, and meta-learning to enhance coordination and adaptability.

Liu *et al.* [6] proposed an active SLAM framework for snake robots that combines multi-sensor fusion with deep reinforcement learning. Their approach emphasizes perception-driven control and mapping accuracy for a single robot, employing centralized sensor fusion. In contrast, our work focuses on multi-agent coordination, with each agent planning and executing policies conditioned on higher-priority agents' trajectories while maintaining decentralized execution.

Wen et al. [7] introduced a meta-reinforcement learning framework for multi-robot path planning using transfer learning to accelerate adaptation across tasks. Their method centralizes meta-learning but retains globally coupled policy updates. By contrast, our H-CMARL-DE framework centralizes only the meta-level

Table 1: Summary of Path Planning Methods.

Aspect	**Dijkstra's**	**A***	**DRRT**	**RL**
Environment Characteristics				
Environment Type	Static	Static	Dynamic, Continuous	Dynamic, Stochastic
Solution Quality				
Optimality	Optimal	Optimal	No guarantee	Depends on training
Multi-Objective	No	Limited	Difficult	Easy
Scalability and Adaptability				
Scalability	Poor	Limited	Good	Good
Adaptability	No	No	Yes	Yes
Computational Aspects				
Computational Cost	High	Low	Moderate	Low runtime, High training
Robustness	Low	Medium	High	High
Learning and Generalization				
Generalization	No	No	No	Yes

optimization—priority order, shaping weights, and subgoal parameters—using Differential Evolution, while policy learning remains fully decentralized. Each agent observes predecessor trajectories but updates its policy independently, reducing communication and preserving scalability.

Recent deep reinforcement learning works, such as the Bidirectional Obstacle Avoidance Enhancement Deep Deterministic Policy Gradient (BOAE-DDPG) [8], the Action-Curiosity-based DRL planner [9], and the Improved DDPG for dense environments [10], focus on enhancing single-agent navigation performance in dynamic or high-density settings. These methods improve local exploration, obstacle avoidance, and convergence stability but lack mechanisms for cooperative coordination or energy-harvesting awareness. In contrast, in this work, using our novel H-CMARL-DE algorithm, we explicitly model spatiotemporal energy fields and energy-aware rewards, integrating them with hierarchical coordination and DE-based meta-optimization to enable scalable, collision-free, and energy-efficient multi-robot navigation.

While recent works has integrated energy consumption into navigation objectives, few studies address path planning for ambiently-powered robots that face temporal and spatial fluctuating energy availability. The related literature falls into two main areas: (A) Energy-Aware path-planning, (B) Energy Harvesting-aware path-planning.

2.2 Energy-Aware Path-Planning

Traditional energy-aware path planning has relied largely on heuristic and model-based strategies. Zhang et al. [11] proposed an energy-efficient variant of the A^* algorithm that incorporates motion primitives tailored to Ackermann-steered autonomous ground vehicles. By integrating vehicle-specific kinematic constraints and energy profiles, the method enables more efficient navigation over extended distances. Dogru and Marques [12] developed an online energy-constrained coverage path planning (ECO-CPP) framework for ground robots. Their algorithm prioritizes battery-aware exploration in real time, ensuring that coverage missions remain feasible within the available energy budget—a critical consideration for autonomous cleaning and surveillance applications. Aiello et al. [13] addressed energy-aware 3D path planning for fixed-wing UAVs, introducing a cost function that accounts for aerodynamic drag and flight dynamics. Although designed for aerial robots, the methodology provides transferable insights for energy modeling in ground-based mobile platforms.

Several recent works have applied deep reinforcement learning to optimize energy-efficient and adaptive path planning for ground robots. One study integrates heuristic corrections into DRL navigation for un-

manned ground vehicles (UGVs), improving policy reliability in unknown terrain and indirectly promoting energy savings [14]. In the context of legged robots, a multiobjective collaborative DRL algorithm has been developed to optimize jumping behavior in bipedal ground robots. Though the focus is motion dynamics rather than energy use, the multiobjective architecture offers a path to integrate energy awareness in locomotion learning [15].

Visca et al. [16] developed an adaptive energy-aware framework tailored for UGVs. Their approach integrates a deep meta-learning model with a meta-adaptive heuristic to achieve energy-efficient path planning, adapting to varying environmental conditions effectively. Focusing on UAV navigation, Bouhamed et al. [17] proposed a DDPG-based strategy designed for obstacle-constrained environments. Their method prioritizes energy efficiency while ensuring high data-rate communication, highlighting the potential of RL in addressing both energy and communication challenges simultaneously. Niaraki et al. [18] introduced a Q-learning framework for UAV path planning under dynamic wind conditions. This approach demonstrated superior adaptability and efficiency compared to traditional complete coverage methods, underscoring the versatility of RL in addressing environmental variability.

2.3 Energy Harvesting-Aware Path-Planning

Recent research has explored energy-harvesting-aware path planning across various robotic platforms, primarily UAVs and ground robots. These studies employ a range of techniques, including sampling-based methods, optimization algorithms, and RL.

Huang et al. [19] introduced an RRT-based approach for energy-efficient path planning of solar-powered UAVs communicating with mobile ground users in urban environments. Their method models urban obstacles and solar irradiance to maximize mission endurance while maintaining communication links. Huang and Savkin [20] later extended this work to a decentralized framework for a team of solar-powered UAVs tasked with collaborative eavesdropping on a moving ground target in urban settings. This follow-up incorporates collision avoidance, limited inter-UAV coordination, and real-time replanning to adapt to the target's trajectory. Ni et al. [21] applied the Soft Actor-Critic (SAC) algorithm to energy-optimal trajectory planning of solar-powered aircraft. Their work jointly optimizes aerodynamic performance and solar-harvesting profiles, demonstrating up to 15% energy savings compared to conventional waypoint-based planning.

Swarm-intelligence methods have also been leveraged for energy-harvesting navigation. Liao et al. [22] used an Ant Colony Optimization (ACO) variant to plan energy-optimal routes for ocean-energy–driven unmanned surface vehicles, integrating spatially varying ocean current maps. Hu and Liu [23] applied an ACO-based algorithm to route planning for solar-powered UAVs, optimizing both path length and expected solar gain in dynamic weather conditions. While these ACO approaches can handle nonlinear environmental models, they typically lack real-time adaptability and explicit multi-robot coordination mechanisms.

Hasankhani et al. [24] proposed an integrated RL-based framework that combines path planning and tracking control for marine current turbines operating in uncertain ocean environments. Using historical current data, their method generates energy-optimal trajectories while maintaining turbine stability against stochastic flow variations. Li et al. [25] surveyed advanced algorithms for guiding solar-powered UAVs, including dynamic programming, interpolation-based solar-gain estimation, and learning-driven strategies. Although Li's work covers a broad spectrum of guidance methods, it does not specifically address multi-agent coordination or ground-based constraints. Ni et al. [26] further extended SAC-based planning to solar-powered aircraft operating in dynamically changing atmospheric conditions. Their study accounts for solar irradiance, wind fields, and gravity effects, demonstrating that RL-trained policies can adapt online to maximize flight endurance under variable weather. Plonski et al. [27] combined dynamic programming with high-resolution solar irradiance maps to plan energy-optimal coverage paths for solar-powered ground robots, validated in ROS/Gazebo simulations. Table 2 provides a brief comparison of the aforementioned EH-aware path planning methods to our work.

Table 2: Energy Harvesting Path Planning Approaches.

Paper	Algorithm	Robot Type	Collision Av.	Harvesting	Multi-Robot	Implementation
[19]	RRT	Fixed-wing UAV	✗	Solar	✗	✗
[20]	DRRT	Multirotor UAV	✓	Solar	✓	✗
[21]	Soft Actor-Critic	Solar UAV	✗	Solar	✗	✗
[22]	Optimized Ant Colony	USV (Marine)	✓	Flow	✗	✗
[23]	Ant Colony Opt.	Solar UAV	✗	Solar	✗	✗
[27]	Dynamic Programming	Ground Robot	✗	Solar	✗	Real Robot
[24]	RL + Tracking	Marine Turbine	✗	Flow	✗	✗
[25]	Dynamic Programming	Solar UAV	✗	Solar	✗	✗
[26]	RL	Solar Aircraft UAV	✗	Wind, Gravity, Solar	✗	✗
This Work	RL / A*	Ground Robots	✓	Multi-source	✓	ROS / Gazebo

3 Problem Formulation

The task of EH-aware robotic navigation involves guiding a robot from a starting position to a goal waypoint within an environment, avoiding obstacles, and maximizing EH efficiency, all while selecting the closest path.

To tackle the challenge of developing an effective navigation strategy that utilizes model-free RL and DRL to adapt to dynamic environments, the problem can be structured using several key components, as detailed below.

3.1 Environment Model

We model hierarchical, energy-harvesting-aware multi-robot navigation on a discrete grid with prioritized planning. Robots plan in a fixed order and each robot conditions only on already-committed trajectories of higher-priority robots.

3.1.1 Grid, Time, and Neighborhood

This subsection defines the spatial and temporal structure of the environment. The workspace is modeled as a discrete two-dimensional grid G of size $M \times N$, where each cell (x, y) represents a possible position that a robot can occupy. Time is also discretized as $T = \{0, 1, \ldots, T_{\max}\}$, forming a time-expanded planning space. Robots can move up to S_i cells per step, plan their actions over a horizon of H future steps, and commit to the first $W \leq H$ steps. The 8-neighborhood $\mathcal{N}_8(x, y)$ defines the set of all adjacent cells a robot may move to in one time step. To handle spatio-temporal planning, we use the time-expanded grid G^{time}, which represents each cell along with its time index.

$$\begin{aligned} &G = \{(x, y) \mid x \in \{1{:}M\},\ y \in \{1{:}N\}\}, \quad N_G = MN, \\ &T = \{0, 1, \ldots, T_{\max}\}, \quad \mathcal{N}_8(x, y) = \text{8-neighborhood}, \\ &S_i \in \mathbb{N}\ (\text{cells/step}), \quad H\ (\text{lookahead}),\ W \leq H\ (\text{commit}). \end{aligned} \tag{1}$$

We use a time-expanded view:

$$\mathrm{G}^{\text{time}} \ = \ \{\, (x, y, \tau) \mid (x, y) \in G,\ \tau \in T \,\}. \tag{2}$$

3.1.2 Robots, Ordering, and Feasibility

Robots are indexed as $R = \{r_1, \ldots, r_n\}$ with a fixed priority order $r_1 \prec r_2 \prec \cdots \prec r_n$. Each robot r_i considers higher-priority robots $\mathcal{H}(i) = \{r_j \mid j < i\}$ when planning its motion to avoid conflicts. The position of robot i at time t is denoted by $p_i^t = (x_i^t, y_i^t)$, and its goal cell is $g_i \in G$. Obstructed cells at each time t are represented by $O_t \subseteq G$. Feasibility requires that a robot never occupies or aims for an obstructed cell,

ensuring both static and dynamic collision avoidance.

$$\begin{aligned} R = \{r_1, \dots, r_n\}, \quad r_1 \prec r_2 \prec \cdots \prec r_n, \\ \mathcal{H}(i) = \{r_j \mid j < i\}, \quad p_i^t = (x_i^t, y_i^t) \in G,\ g_i \in G. \end{aligned} \tag{3}$$

Let $O_t \subseteq G$ be obstructed cells at time t. Feasible occupancy:

$$p_i^t \in G \setminus O_t \ \ \forall t \in T, \qquad g_i \in G \setminus \bigcup_{t \in T} O_t. \tag{4}$$

3.1.3 Paths and Hierarchical

Each robot r_i follows a discrete path $P_i = (p_i^0, \dots, p_i^{T_i})$, where $n_{P_i} = T_i + 1$ is the number of time steps in the path. A time-indexed reservation table $R^\tau \subseteq G$ keeps track of all occupied cells at each time step τ. When robot i plans its next moves, it considers reserved cells of higher-priority robots $\mathcal{H}(i)$ as well as obstacles O_τ. The combined set of unavailable cells at time τ is $C_i^\tau = O_\tau \cup R_{\text{pred}}^\tau$. At each planning time t, robot i accesses the predicted or committed future positions of higher-priority robots within the lookahead window H, and then commits its own next W steps to the reservation table to ensure consistency across agents.

$$P_i = \big(p_i^0, \dots, p_i^{T_i}\big), \quad n_{P_i} = T_i + 1. \tag{5}$$

We maintain a time-indexed reservation table $R^\tau \subseteq G$ at each step τ. For robot i, the predecessor reservations are:

$$R_{\text{pred}}^\tau \ = \ \{\, p_j^\tau \mid j \in \mathcal{H}(i) \,\}, \qquad C_i^\tau \ = \ O_\tau \cup R_{\text{pred}}^\tau \subseteq G. \tag{6}$$

At planning time t, each $r_j \in \mathcal{H}(i)$ shares planned/committed cells over horizon H:

$$\widehat{P}_{\mathcal{H}(i)}^{t:t+H} = \{\, p_j^\tau \mid j \in \mathcal{H}(i),\ \tau \in \{t, \dots, t+H\} \,\}. \tag{7}$$

Robot i publishes (commits) its first W planned steps to the table:

$$\begin{aligned} \text{Commit}_i(t{:}t{+}W{-}1) \ = \ \{\, p_i^\tau \mid \tau \in \{t, \dots, t{+}W{-}1\} \,\}, \\ R^\tau \leftarrow R^\tau \cup \{p_i^\tau\}. \end{aligned} \tag{8}$$

Collision constraints (time-step based):

$$p_i^\tau \notin C_i^\tau \ \ \forall \tau, \tag{9}$$

$$(p_i^\tau, p_i^{\tau+1}) \neq (p_j^{\tau+1}, p_j^\tau) \ \ \forall j \in \mathcal{H}(i),\ \tau < T_{\max}. \tag{10}$$

Projection rule (enforced each step): if a chosen move would violate (9) or (10), set $p_i^{\tau+1} \leftarrow p_i^\tau$ (stay) and reserve p_i^τ at $\tau{+}1$.

$$p_i^{\tau+1} \ \leftarrow \ \begin{cases} p_i^\tau, & \text{if } p_i^{\tau+1} \in C_i^{\tau+1} \text{ or edge-swap}, \\ p_i^\tau + S_i\, \Delta(A_{i,\tau}), & \text{otherwise}. \end{cases} \tag{11}$$

This makes reservations strictly *per time-step* on the time-expanded graph G^{time}, ensuring no same-time co-occupancy and no head-on swaps. The commit window W provides lookahead stability; after executing one step, agents replan, re-commit, and update R^τ.

3.1.4 Dynamic Obstacles

We model the motion of dynamic obstacles as a stochastic process that captures diverse and realistic movement behaviors. Each obstacle follows a parametric Markov model with multiple motion modes, allowing the environment to exhibit heterogeneous dynamics during both training and evaluation.

$$\begin{aligned} &O_{t+1} \sim p_\theta(\cdot \mid O_t), \\ &o_k^{t+1} = o_k^t + v_k^t + \epsilon_k^t, \qquad v_k^t \sim \mathcal{D}_\theta, \quad \epsilon_k^t \sim \mathcal{N}(0, \Sigma), \\ &\mathcal{D}_\theta : \text{mixture of } \{\text{static, constant-velocity, lane/flow-following, stop-and-go}\}. \end{aligned} \tag{12}$$

In this formulation, each obstacle o_k updates its position based on a nominal velocity v_k^t sampled from the motion-mode distribution $\mathcal{D}_\theta$, plus small Gaussian noise ϵ_k^t representing stochastic perturbations. The parameter θ defines environment-level properties such as obstacle densities, preferred speeds, and flow-field directions.

During training, these parameters are randomized as $\theta \sim \Pi_{\text{train}}$ to encourage robustness through domain randomization. For evaluation, we use unseen out-of-distribution settings $\theta \sim \Pi_{\text{OOD}}$ to test generalization under new obstacle motion regimes. Performance metrics include overall success rate, number of collisions per 1000 steps, average delay, and path stretch relative to the optimal trajectory.

3.1.5 Spatiotemporal Energy Harvesting

We introduce a time-varying field with moving sources, persistence, and normalization at each t:

$$\begin{aligned} \overline{EH}_i^{(x,y),t} &= P_i^{\text{harv}}(x,y,t)\, T_i^{\text{harv}}(x,y,t), \\ EH^{(x,y),t} &= \frac{\overline{EH}_i^{(x,y),t}}{EH_{\max}^G(t)}, \quad EH_{\max}^G(t) = \max_{(u,v)\in G} \overline{EH}_i^{(u,v),t}. \end{aligned} \tag{13}$$

Source-based evolution:

$$\begin{aligned} \mathcal{S}_t &= \{(c_k(t), s_k(t))\}_{k=1}^K, \\ EH_{\text{base}}^{(x,y),t} &= \sum_{k=1}^K s_k(t)\, K_\sigma\big(\|(x,y) - c_k(t)\|_2\big), \\ EH^{(x,y),t+1} &= (1-\rho)\, \widetilde{EH}_{\text{base}}^{(x,y),t+1} + \rho\, EH^{(x,y),t} + \varepsilon^t, \end{aligned} \tag{14}$$

where K_σ is a spatial kernel, $\rho \in [0,1)$ controls persistence, ε^t is small noise, and $\widetilde{EH}_{\text{base}}$ is normalized to $[0,1]$. Optional advection by a flow field u:

$$EH^{(x,y),t+1} \leftarrow EH^{(x,y),t+1} - \Delta t\, [u \cdot \nabla EH] + D\, \Delta EH, \tag{15}$$

4 Baselines and Proposed Methods

4.1 Hierarchically Conditioned Multi-Agent RL with Meta-DE (H-CMARL-DE)

4.1.1 Overview

We propose **H-CMARL-DE** (Hierarchically Conditioned Multi-Agent Reinforcement Learning with Differential Evolution), which integrates:

- **Meta-level centralized optimization**: Differential Evolution (DE) [28] co-optimizes execution order, subgoal/commit parameters, and reward shaping weights.

- **Hierarchically conditioned per-agent policy learning**: Each agent's policy gradient is computed using its own state *and* the planned trajectories, start, and goal of higher-priority agents.
- **Decentralized execution**: Agents execute sequentially based on hierarchical time-space reservations, guaranteeing collision-free motion.

Unlike classical CTDE, which centralizes policy gradients, H-CMARL-DE centralizes only meta-variables; per-agent policies remain decentralized but hierarchically conditioned.

Key novelties

- Hierarchical execution with time-indexed reservations reduces non-stationarity and ensures safety.
- DE optimizes global coordination variables without requiring centralized gradient updates.
- Policy learning is hierarchically conditioned on predecessors' planned paths, which creates a structured dependency while avoiding full joint learning.

4.1.2 Hierarchical Path-Conditioned Policy Learning

Each agent i observes a state vector including predecessors' planned trajectories:

$$s_{i,\tau} = \big(p_i^{\tau}, g_i, O_\tau, R^\tau, EH_\tau^G, \{p_j^{\tau:t+H}\}_{j<i}\big) \tag{16}$$

The agent then selects an action $a_{i,\tau} = \pi_{\theta_i}(s_{i,\tau})$, gradient updates are per-agent and use this hierarchical conditioning.

4.1.3 Action (A) and Transition

$$\begin{aligned} A = \{&\text{stay},\ \text{up},\ \text{up-right},\ \text{right},\ \text{down-right},\\ &\text{down},\ \text{down-left},\ \text{left},\ \text{up-left}\}, \end{aligned} \tag{17}$$

subject to (10); infeasible actions are projected to stay.

4.1.4 Reward (R): Safety, Goal, Efficiency, Harvest

Base terms:

$$\begin{aligned} R_{\text{goal}} &= \begin{cases} +r_{\text{goal}}, & p_i^{t+1} = g_i, \\ 0, & \text{otherwise}, \end{cases} \quad R_{\text{coll}} = \begin{cases} -r_{\text{coll}}, & \text{if 10 violates}, \\ 0, & \text{otherwise}, \end{cases} \\ R_{\text{move}} &= -r_{\text{move}}, \qquad R_{\text{EH}} = \eta_t\, EH\big(p_i^{t+1}, t\big). \end{aligned} \tag{18}$$

Per-step reward:

$$R = \begin{cases} r_{\text{goal}} - r_{\text{move}} + \eta\, EH, & p_i^{t+1} = g_i, \\ -r_{\text{coll}} - r_{\text{move}} + \eta\, EH, & \text{if infeasible (projected)}, \\ -r_{\text{move}} + \eta_t\, EH, & \text{otherwise}. \end{cases} \tag{19}$$

Note: Because $s_{i,\tau}$ includes other agents' plans, the resulting process can be viewed as a *Hierarchically Conditioned MDP (HMDP)*, where the next-state distribution depends not only on the current state and action but also on the exogenous planned trajectories of higher-priority agents. While this relaxes the classical Markov assumption, similar structured dependencies are found in hierarchical RL frameworks [29, 30].

Algorithm 1 H-CMARL-DE Execution: Hierarchical Reservation with Conditioned Policies

Input: Meta-order π, horizon H, commit window W, per-agent policies $\{\pi_{\theta_i}\}$
$R^t \leftarrow \emptyset$
for $i = 1$ to n in order π **do**
  for $\tau = t$ to $t + H$ **do**
    $s_{i,\tau} \leftarrow (p_i^\tau, g_i, O_\tau, R^\tau, EH_\tau^G, \{p_j^{\tau:t+H}\}_{j<i})$
    $a_{i,\tau} \leftarrow \pi_{\theta_i}(s_{i,\tau})$; project if violating constraints
    $p_i^{\tau+1} \leftarrow p_i^\tau + S_i \cdot \Delta(a_{i,\tau})$
    Update energy $E_i^{\tau+1}$
    Reserve cell: $R^{\tau+1} \leftarrow R^{\tau+1} \cup \{p_i^{\tau+1}\}$
  end for
  Commit first W steps
end for=0

4.1.5 Meta-Level DE Optimization and Hierarchical Conditioning

Each robot r_i learns a decentralized policy $\pi_{\theta_i}(a_i \mid S_{i,t})$, while a *meta-level optimizer* adjusts global coordination and reward-shaping parameters consistent with the environment model in Sec. 3.1. The meta-parameter vector is defined as

$$\xi = [\pi,\, H,\, W,\, \eta,\, \kappa, \rho,\, D], \tag{20}$$

where π denotes the learned priority ordering of robots ($r_1 \prec r_2 \prec \cdots \prec r_n$), H is the lookahead horizon, and W is the commit window length. The coefficient η scales the time-varying reward associated with energy harvesting, while κ weights the delay components in the global objective. The parameters ρ and D govern the temporal persistence and spatial diffusion of the spatiotemporal energy-harvesting field, determining how harvested energy evolves over time and space under environmental uncertainty.
Together, these parameters encode both *coordination structure* (via π, H, W) and *environment-aware adaptability* (via η, ρ, κ,D).
Role of DE: Differential Evolution (DE) operates at the *meta level*, outside the inner-loop RL training of individual robots. It samples a population of candidate ξ configurations, executes episodic rollouts of decentralized policies under each $\xi^{(k)}$, evaluates their fitness via:

$$\begin{aligned} J(\xi) = {} & w_1 \,\mathrm{Succ} - w_2 \,\mathrm{Coll} - w_3 \,\mathrm{Stretch} \\ & - w_4 \,\mathrm{Delay} + w_5 \,\mathrm{EH} \end{aligned} \tag{21}$$

and evolves the population using DE mutation–crossover–selection steps to identify the highest-performing $\xi^\star$. Importantly, DE only adjusts the coordination structure, reward balance, and safety biases for the decentralized learners; it does **not** update the PPO parameters θ_i directly.
Definitions of terms:

- **Succ (Success):** Fraction of robots that reach their goals within the planning horizon:

$$\mathrm{Succ} = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\{p_i^{T_i} = g_i\}.$$

- **Coll (Collision):** Number of collisions with obstacles or higher-priority robots along the trajectory:

$$\mathrm{Coll} = \sum_{i=1}^{n} \sum_{t=0}^{T_i} \mathbf{1}\{p_i^t \in O_t \text{ or } p_i^t \in R_{\text{pred}}^t\}.$$

Algorithm 2 Differential Evolution for Meta-Level Coordination

```
Initialize population {ξ^(k)}_{k=1}^K
for generation = 1 : G do
    for each candidate ξ^(k) do
        Rollout episodes using decentralized policies under ξ^(k)
        Evaluate J(ξ^(k))
    end for
    for each candidate ξ^(k) do
        Mutate and crossover to generate ξ̃^(k) {Explores new coordination configurations only}
        Selection: ξ^(k) ← arg max{J(ξ^(k)), J(ξ̃^(k))}
    end for
end for
Return best meta-parameter configuration ξ* =0
```

- **Stretch:** Ratio of executed path length to shortest path length, averaged over all robots:

$$\text{Stretch} = \frac{1}{n}\sum_{i=1}^{n}\frac{\text{length}(P_i)}{\text{length}(P_i^{\min})}.$$

- **Delay:** Extra steps taken beyond the minimum path length:

$$\text{Delay} = \frac{1}{n}\sum_{i=1}^{n}\left(T_i - T_i^{\min}\right).$$

- **EH-Yield (Energy Harvesting):** Cumulative energy collected along trajectories:

$$\text{EH} = \sum_{i=1}^{n}\sum_{t=0}^{T_i} EH_i^t.$$

The weights $w_1, \ldots, w_6$ normalize and balance different objectives. Positive terms (Succ, EH-Yield) are rewards to maximize, while negative terms (Coll, Stretch, Delay) are penalties to minimize. DE optimizes ξ to maximize $J(\xi)$ over episodic rollouts, without interfering with the decentralized learning updates of the individual PPO agents.

Interpretation:

- The inner-loop decentralized RL treats higher-priority trajectories as dynamic constraints.
- The outer-loop DE tunes ξ to minimize global cost $J(\xi)$ across episodes.
- The resulting hierarchy balances tractable planning, collision-free safety, and adaptive energy harvesting, while preserving decentralized learning autonomy.

4.1.6 Theoretical Justification

Collision avoidance: Hierarchically conditioned execution with time-indexed reservations guarantees that no two robots co-occupy the same cell or swap edges. Formally, for agent i at time τ:

$$p_i^\tau \notin R^\tau \quad \wedge \quad (p_i^\tau, p_i^{\tau+1}) \neq (p_j^{\tau+1}, p_j^\tau) \Rightarrow \text{no collision.} \tag{22}$$

Hierarchical conditioning and non-stationarity mitigation: Each agent i acts in a constrained MDP that is conditioned on the planned trajectories, start, and goal of all higher-priority agents $j < i$. The lower-triangular dependency structure ensures that the predecessors' actions can be treated as exogenous by agent i, and successors do not affect i's feasible action space. As a result, the effective learning environment for each agent becomes more stable from its perspective compared to fully joint updates in CTDE. Importantly, **the global environment remains non-stationary** due to ongoing learning and coordination among agents [31, 32], but the hierarchical conditioning serves to reduce the non-stationarity faced by each individual agent.

Computational complexity: Classical CTDE approaches require reasoning over joint action spaces during training, leading to multiplicative scaling with the number of agents:

$$\mathcal{O}\Big(\prod_{i=1}^{n} |A_i|\Big),$$

where n denotes the number of agents and $|A_i|$ is the size of the action space of agent i. Centralized critics typically introduce additional $\mathcal{O}(n^2)$ coupling in gradient updates.

In contrast, H-CMARL-DE performs **hierarchically conditioned decentralized learning**, where each agent operates only with respect to its own state and the planned trajectories of higher-priority agents. This yields an **additive** execution-time complexity:

$$\mathcal{O}\Big(\sum_{i=1}^{n} |A_i| + nH\Big),$$

where H is the planning horizon. Reservation checks incur only linear overhead, ensuring that execution remains **real-time scalable** as n increases.

Training-time considerations: The meta-level DE introduces an additional outer-loop optimization cost; however, this cost is (i) inherently parallelizable, (ii) executed intermittently rather than at every policy update, and (iii) decoupled from joint gradient backpropagation over the full multi-agent action space. Moreover, hierarchical conditioning stabilizes each agent's learning environment, resulting in fewer policy updates to reach coordinated behavior [33]. Recent work has shown that DE can be made significantly faster in wall-clock performance; for example, DEHB achieves up to 1000 times faster search by allocating computation more efficiently [34].

Optimality: Prioritized planning under a fixed execution order is not globally optimal in general. However, DE continuously optimizes the meta-variables (priority ordering, horizon, commit window, and reward shaping weights) to narrow this gap while preserving strict collision avoidance guarantees. This results in near-optimal coordination without centralized gradient coupling.

4.2 Advantage Actor-Critic

A model-free, synchronous algorithm combining policy-based and value-based methods. It introduces the advantage function as:

$$A(s_t, a_t) = R(s_t, a_t) + \gamma V(s_{t+1}) - V(s_t) \tag{23}$$

Where the policy is updated by:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a_t|s_t) A(s_t, a_t)\right] \tag{24}$$

and the value function is updated as follows:

$$V_\phi(s_t) \leftarrow V_\phi(s_t) + \alpha \delta_t \tag{25}$$

4.3 Proximal Policy Optimization

A policy gradient method optimizing π_θ directly. The objective function is:

$$L^{\text{PPO}}(\theta) = \mathbb{E}\Big[\min\big(r_t(\theta)\,\hat{A}_t, \\ \text{clip}\big(r_t(\theta), 1-\epsilon, 1+\epsilon\big)\,\hat{A}_t\big)\Big] \tag{26}$$

where $r_t(\theta)$ is the action ratio, and $\hat{A}_t$ is the advantage estimate with a clipping parameter ϵ.

4.4 Collision-Free A*-EH (Reservation-Based Prioritized Planning)

We extend A*-EH to a multi-robot, time-indexed setting that guarantees collision freedom via a reservation table on a time-expanded graph. Robots plan in a fixed priority order, and each robot conditions only on the already-committed trajectories of higher-priority robots. This preserves scalability and enables online re-planning under dynamic obstacles and spatiotemporal energy fields.

Time-Expanded Graph and Reservations: Let $G = \{(x, y)\}$ be the grid, $T = \{0, 1, \ldots, T_{\max}\}$ discrete time, and

$$G^{\text{time}} \;=\; \{\,(x, y, \tau) \mid (x, y) \in G,\ \tau \in T\,\}.$$

Let $R = \{r_1, \ldots, r_n\}$ with fixed order $r_1 \prec \cdots \prec r_n$. We maintain time-indexed reservations for vertices and edges:

$$R^\tau \subseteq G \quad \text{(reserved vertices at time } \tau\text{)}, \tag{27}$$

$$E^{\tau \to \tau+1} \subseteq G \times G \quad \text{(reserved oriented edges)}. \tag{28}$$

At planning time t, robot i observes the committed occupancies of its predecessors over a horizon H:

$$R^\tau_{\text{pred}} = \{\, p^\tau_j \mid j < i \,\}, \tag{29}$$

$$E^{\tau \to \tau+1}_{\text{pred}} = \{\, (p^\tau_j, p^{\tau+1}_j) \mid j < i \,\}, \tag{30}$$

$$C^\tau_i \;= O_\tau \cup R^\tau_{\text{pred}}. \tag{31}$$

where O_τ are obstructed cells at time τ. Each robot plans over $[t, t{+}H]$ and commits its first $W \leq H$ steps; after executing one step, all robots replan in a rolling-horizon fashion.

EH-Aware Cost and Heuristic: We employ A*-EH on the time-augmented graph G^{time}, incorporating a per-step penalty for "stay" actions to preserve admissibility and consistency.
Step Cost. For a transition $(x, y, \tau) \to (x', y', \tau{+}1)$, define

$$c_{\text{step}} = \begin{cases} 1, & (x', y') = (x, y), \\ \|(x', y') - (x, y)\|_2, & \text{otherwise.} \end{cases} \tag{32}$$

Equivalently,

$$c_{\text{step}} = \max\big\{1,\ \|(x', y') - (x, y)\|_2\big\}. \tag{33}$$

EH-Aware Cost: The accumulated cost at $(x', y', \tau{+}1)$ is

$$g_{\text{EH}} = g(x, y, \tau) + c_{\text{step}} - \alpha\, EH^{(x',y'),\tau+1}, \tag{34}$$

where $\alpha \in [0, 1]$ and $EH \in [0, 1]$.
EH-Aware Heuristic: We use an admissible and consistent heuristic

$$h_{\text{EH}} = (1 - \alpha)\, D_\infty\big((x, y), g\big), \tag{35}$$

since each step increases cost by at least $(1 - \alpha)$, while the Chebyshev distance D_∞ decreases by at most 1 per step.
A stronger (yet still admissible) alternative is the octile heuristic, scaled by $(1 - \alpha)$.

Feasibility Constraints (Collision Freedom): While expanding neighbors on G^{time}, enforce:

$$\text{(vertex)} \quad (x', y') \notin C_i^{\tau+1} \quad \forall \tau, \tag{36}$$

$$\text{(edge)} \quad \big((x, y), (x', y')\big) \notin E_{\text{pred}}^{\tau\to\tau+1} \quad \forall \tau, \tag{37}$$

$$\text{(no head-on swap)} \quad \big((x, y), (x', y')\big) \neq \big(p_j^{\tau+1}, p_j^{\tau}\big) \quad \forall j < i. \tag{38}$$

A stay action is included. At execution time, if the chosen next state would violate (36)–(38) due to unmodeled changes, apply a projection rule: stay-put and reserve the current cell at the next time.

Collision-Free Guarantee: Suppose each robot i plans using Algorithm 3, enforcing (36)–(38), and commits its first W steps to R^τ and $E^{\tau\to\tau+1}$. Then for all τ and all $i > j$: (i) $p_i^\tau \neq p_j^\tau$ (no same-time co-occupancy), and (ii) $(p_i^\tau, p_i^{\tau+1}) \neq (p_j^{\tau+1}, p_j^\tau)$ (no head-on swap). By induction on priority and time. For $j < i$, p_i^τ is constructed to avoid C_i^τ and edge conflicts with robot j's committed moves; thus $p_i^\tau \notin R^\tau$ and no swap occurs. The projection rule preserves these invariants under execution-time deviations.

Optimality and Complexity: For fixed R^τ, $E^{\tau\to\tau+1}$, O_τ, and EH, CF-A*-EH yields an A*-optimal path for robot i under the composite cost in (34) with the admissible and consistent heuristic h_{EH}. Global optimality across all robots cannot be guaranteed because trajectories are planned sequentially under a fixed priority order, so earlier paths constrain later ones. Nevertheless, the scheme remains complete and deadlock-free with respect to the reservation-constrained search on G^{time}. Each plan requires $O(H\,|G|\log|G|)$

Algorithm 3 Collision-Free Prioritized A*-EH (CF-A*-EH)

Require: Priority order $r_1 \prec \cdots \prec r_n$, starts $\{p_i^0\}$, goals $\{g_i\}$, grid G, dynamic obstacles $\{O_\tau\}$, EH field $\{EH^{(x,y),\tau}\}$, $\alpha \in [0,1]$, horizon H, commit window $W \leq H$

Ensure: Reservation-consistent plans; per-step commits to R^τ and $E^{\tau\to\tau+1}$

Initialize $R^\tau \leftarrow \emptyset$, $E^{\tau\to\tau+1} \leftarrow \emptyset$ for all $\tau \in T$
for planning rounds $t = 0, 1, \ldots$ until all robots reach goals **do**
  for $i = 1$ to n **do**
    $R^\tau_{\text{pred}} \leftarrow \{p_j^\tau \mid j < i,\ \tau \in [t, t{+}H]\}$; $E^{\tau\to\tau+1}_{\text{pred}} \leftarrow \{(p_j^\tau, p_j^{\tau+1}) \mid j < i,\ \tau \in [t, t{+}H{-}1]\}$
    $C_i^\tau \leftarrow O_\tau \cup R^\tau_{\text{pred}}$; start $s_i = (x_i^t, y_i^t, t)$; goal set $\mathcal{G}_i = \{(g_i, \tau) \mid \tau \in [t, t{+}H]\}$
    Open $\leftarrow \{s_i\}$; $g \leftarrow \infty$; $\pi \leftarrow$ None; $g(s_i) \leftarrow 0$; $f(s_i) \leftarrow h_{\text{EH}}(p_i^t, g_i)$
    while Open not empty **do**
      Pop $u{=}(x, y, \tau)$ with minimum f
      if $(x, y) = g_i$ **then**
        break
      end if
      for each neighbor $v{=}(x', y', \tau{+}1)$ in $\mathcal{N}_8 \cup \{\text{stay}\}$ **do**
        if $(x', y') \in C_i^{\tau+1}$ **then**
          continue
        end if{no same-time co-occupancy}
        if $\big((x, y), (x', y')\big) \in E^{\tau\to\tau+1}_{\text{pred}}$ **then**
          continue
        end if{no edge conflict}
        if $\exists j < i$ with $(x, y) = p_j^{\tau+1}$ and $(x', y') = p_j^\tau$ **then**
          continue
        end if{no head-on swap}
        $g_{\text{new}} \leftarrow g(x, y, \tau) + c_{\text{step}}\big((x, y) \to (x', y')\big) - \alpha\, EH^{(x',y'),\tau+1}$
        if $g_{\text{new}} < g(v)$ **then**
          $g(v) \leftarrow g_{\text{new}}$; $f(v) \leftarrow g_{\text{new}} + h_{\text{EH}}((x', y'), g_i)$; $\pi(v) \leftarrow u$; push v to Open
        end if
      end for
    end while
    Reconstruct $\widehat{P}_i^{t:t+H}$ via π; commit first W steps:
    For $k \in [t, t{+}W{-}1]$: add p_i^k to R^k and (p_i^k, p_i^{k+1}) to $E^{k\to k+1}$
  end for
  Execute one step for all robots (in priority order) with projection rule if needed; advance to $t{+}1$
end for=0

per robot (8-neighborhood plus stay), yielding $O(n\,H\,|G| \log |G|)$ per replan round; edge checks add only constant overhead.

Dynamic Replanning and Probabilistic Completeness: To handle time-varying obstacles and energy fields, CF-A*-EH uses a receding-horizon replanning strategy with frequency ν. Assume that the obstacle dynamics are ergodic, meaning that over time the environment will eventually present all feasible configurations. Further, suppose that within each horizon of H steps there exists (with probability at least $p_0 > 0$) a static snapshot that admits a collision-free path to the goal. Then, across independent or sufficiently mixing horizon windows, the probability of *not* encountering a feasible snapshot after K replanning steps is bounded by $\Pr[\text{failure}] \leq (1 - p_0)^K$, which vanishes as $K \to \infty$. Consequently, the receding-horizon CF-A*-EH scheme is probabilistically complete under ergodic, time-varying environmental dynamics.

5 Performance Metrics and Results

5.1 Performance Metrics

To evaluate the effectiveness of the navigation strategies, several performance metrics are considered:

- **Success Rate**: This metric quantifies how often each robot successfully reaches its goal ($n_{success}$)within a given number of attempts ($n_{attempts}$):

$$\text{Success Rate} = \frac{n_{\text{success}}}{n_{\text{attempts}}} \times 100\% \tag{39}$$

  where a higher success rate indicates better navigation strategies.

- **Mission Completion Time (MCT)**: This metric represents the average time required to complete successful path planning attempts:

$$\text{MCT} = \frac{1}{n_{\text{success}}} \sum_{i=1}^{n_{\text{success}}} t_{P_r}^{(i)} \tag{40}$$

- **Average Path Length (APL)**: This metric calculates the average path length to reach the goal across all successful attempts:

$$APL = \frac{1}{n_{\text{success}}} \sum_{i=1}^{n_{\text{success}}} \left(\sum_{t=1}^{n_{P_r(i)}-1} \sqrt{(x_r^{t+1}{}_{(i)} - x_{r(i)}^t)^2} + (y_r^{t+1}{}_{(i)} - y_{r(i)}^t)^2 \right) \tag{41}$$

- **Average Energy Harvested (AEH)**: This measures how effectively robots harvest energy during their navigation tasks:

$$AEH = \frac{1}{n_{\text{success}}} \sum_{i=1}^{n_{\text{success}}} \sum_{t=1}^{n_{P_r(i)}} EH_r^{G_{x_r^{t(i)}, y_r^{t(i)}}} \tag{42}$$

  where n_{P_r} is the number of grid steps of the path P_r.

  A higher EH efficiency implies better utilization of available resources, enabling longer operational times.

- **Average Closest Path Length ($ACPL$)**: The $P_{j_{\min}}$ represents the average length of the shortest (i.e., closest-cost) paths between start p_r^0 and goal g_r positions, and n_p is the number of steps in the path.

  The length of this optimal path is computed using the Euclidean distance between consecutive steps, where (x_i,y_i) are each grid position of G_{x_i,y_i}:

$$L_{j_{\min}} = \sum_{i=1}^{n_p-1} \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2} \tag{43}$$

  Finally, the $ACPL$ is defined as the average of these minimum-length paths over all successful planning instances for different start p_r^0 and goal g_r positions:

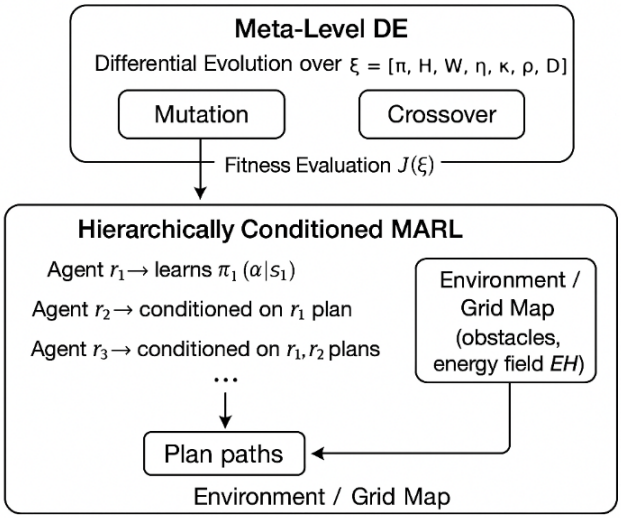


Figure 1: The overview of the learning process. This Figure shows the different steps of EH-aware path planning. Step 1 shows how the environment is modeled using Gymnasium. The second step shows how the model is trained. To train the model the RL algorithm should interact with the developed model of the environment and robots in Gymnasium and based on the gained rewards train the model. In the third step, the completely trained model is used to execute and test the EH-aware path planning in Gymnasium and gain the results.

$$ACPL = \frac{1}{n_{\text{success}}} \sum_{i=1}^{n_{\text{success}}} L_{j_{\min}}^{(i)} \tag{44}$$

Note that in this study, each $L_{j_{\min}}^{(i)}$ is the shortest path length obtained using the standard A* algorithm [5].

- **Energy-Aware Metrics**
  - **Energy Harvesting Index (EHI):** Measures the average harvestable energy in the environment:
  $$EHI = \frac{\sum_{\forall G_{x,y}} EH^{G_{x,y}}}{M \cdot N} \tag{45}$$
  where $M \times N$ is the size of the grid.
  - **Path Length (PL) Efficiency:** Ratio of the Average Path Length (APL) to the Average Closest Path Length (ACPL). A value closer to 1 indicates a more efficient path:
  $$\text{PL Efficiency} = \frac{\text{APL}}{\text{ACPL}}$$
  - **Energy Harvesting (EH) Efficiency:** Measures the average harvested energy per unit path length:
  $$\text{EH Efficiency} = \frac{\text{AEH}}{\text{APL}}$$

5.2 Results

This section presents the results obtained from running various test scenarios using PPO, and A2C. The results include information on navigation performance and computational efficiency, as captured through our evaluation criteria (EC1 and EC2). The learning process was conducted using the OpenAI Gymnasium environment [35], which provides a versatile platform for reinforcement learning experiments. For algorithm implementation, the PPO and A2C functions from the Stable-Baselines3 library [36] were employed. Figure 1 provides an overview of the overall learning process.

5.2.1 EC1: Navigation Performance

The first evaluation criterion (EC1) focuses on navigation performance, particularly examining the average path length and the amount of harvested energy. Tests were conducted under both static and dynamic conditions to assess each algorithm's adaptability and energy-awareness.

The reward function (Equation 18) combines sparse components (goal achievement and collision penalties) with dense rewards for energy harvesting. This composite design increases the risk of agents settling into local optima, where energy harvesting outweighs goal-reaching, especially at higher η values. To counteract this, a three-stage curriculum learning process was used. In the first phase, agents learned fundamental path-planning without dynamic elements. The second phase introduced dynamic obstacles, focusing on collision avoidance. The final phase incorporated energy-harvesting awareness, progressively tuning η to achieve a balance between navigation efficiency and energy collection. For higher-dimensional grids (50×50 and 100×100), transfer learning was applied to accelerate convergence by initializing agents with weights obtained from lower-resolution environments.

Environment Dynamics: The environment consists of static obstacles, dynamic obstacles, and a time-varying energy field $EH^{(x,y),t}$.
Static obstacles: At the start of each episode, 10% of grid cells are selected uniformly at random and marked as static obstacles. Their positions remain fixed throughout the episode.
Dynamic obstacles: A fixed set of $|\{o_k\}| = 5$ dynamic obstacles move according to a parametric Markov motion model p_θ. At each step, obstacle velocities v_k^t are sampled from a mixture distribution $\mathcal{D}_\theta$ (static, constant-velocity, flow-aligned, stop-and-go modes), with added Gaussian noise $v_k^t \leftarrow v_k^t + \epsilon$, $\epsilon \sim \mathcal{N}(0, \Sigma)$. The parameter vector θ controlling mixture weights and speeds is:

$$\theta \sim \Pi_{\text{train}} \text{ during training}, \qquad \theta \sim \Pi_{\text{OOD}} \text{ during evaluation}.$$

Speed magnitudes are sampled per episode:

$$\|v_k^t\| \sim (0, 2) \text{ (train)}, \quad \|v_k^t\| \sim \ (2, 3) \text{ (test)}.$$

Energy field: The field is generated from $K = 3$ moving sources and evolves via spatial smoothing and temporal persistence. The following parameters are explicitly sampled once per episode:

$$\rho \sim (0.5, 0.95), \qquad D \sim (0, 0.1), \qquad \|u\| \sim (0, 0.5).$$

These control, respectively, temporal decay, spatial diffusion, and flow advection strength.
Table 3 illustrates the environment and algorithm hyperparameters used for this section's simulation.

Table 3: Environment and algorithm hyperparameters.

Parameter	**Value / Range / Percent**
Environment	
Grid size	10×10, 50×50, 100×100
$R_{\text{collision}}$	-5
R_{goal}	50
R_{move}	-1
η	3
Static obstacle fraction	10% of total grid size
Dynamic obstacle count $\lvert\{o_k\}\rvert$	5
p_θ	Parametric Markov model
$\mathcal{D}_\theta$	Mixed
$\lVert v_k^t \rVert$	Train $\sim (0, 2)$, Test $\sim (2, 3)$
Σ	Diagonal, small (fixed)
θ sampling	Π_{train} / Π_{OOD}
$\lVert u \rVert$	$\sim (0, 0.5)$
K	3
ρ	$\sim (0.5, 0.95)$
D	$\sim (0, 0.1)$
RL hyperparameters	
Entropy coef (ent_coef)	0.1
Learning rate (α_l)	3×10^{-4}
n_{steps}	1000
Batch size	256
γ	0.95
λ	95%
Value function coef (vf_coef)	0.3
CFA* parameters	
Energy-weighting factor (α)	0.5

Comparison framework: For benchmarking, two heuristic algorithms (CFA* and CFA*EH) and two dynamic learning-based models (CTDE and H-CMARL-DE) were compared. Heuristic methods (CFA* and CFA*EH) rely on precomputation and static planning and cannot adapt during execution, whereas CTDE and H-CMARL-DE dynamically adjust their navigation and energy strategies in real time using learned policies. Figure 2 and 3 demonstrate the test results for the aforementioned methods.

Performance evaluation: Results averaged over 10 trials per configuration show that while CFA* achieves the shortest path length, it fails to adapt when environmental factors change; CFA*EH improves static energy performance but remains rigid. CTDE and H-CMARL-DE with $\eta = 3$ are energy harvesting aware and, based on their learning method, adaptively respond to moving obstacles and shifting energy distributions, maintaining high success rates under non-stationary conditions.

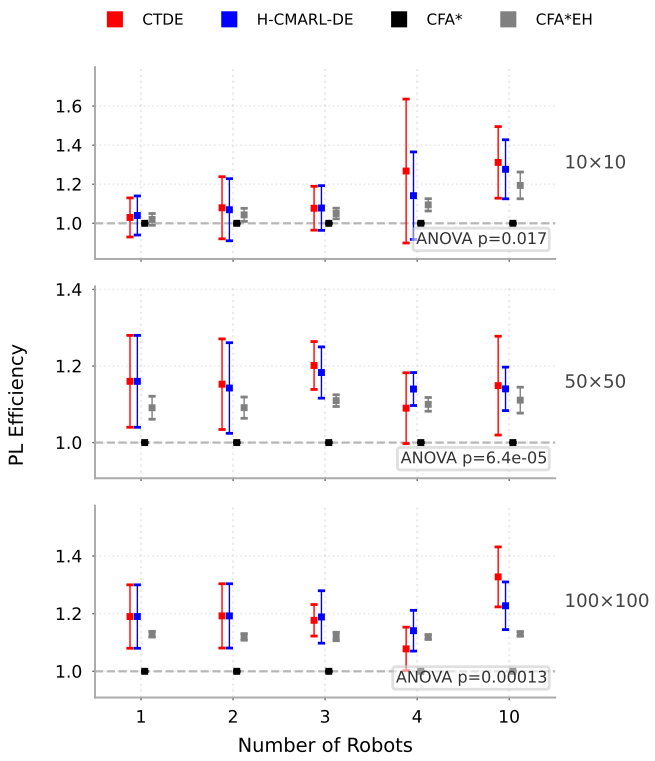


Figure 2: Path-length efficiency comparison. CTDE and H-CMARL-DE incur modest path-length increases vs. static heuristics, with H-CMARL-DE matching CTDE while boosting energy gains and robustness.

Energy harvesting efficiency: H-CMARL-DE attains energy-harvesting efficiency comparable to or better than CFA*EH while operating dynamically rather than via static preplanning. Compared to CTDE, H-CMARL-DE improves energy-harvesting efficiency by an average of 7.3 %, highlighting the advantage of its hierarchical control mechanism in balancing global energy cues and local reactive control to maximize harvesting without sacrificing task completion.

Path-length efficiency: CTDE and H-CMARL-DE incur a modest path-length increase relative to static heuristics, reflecting dynamic detours to avoid collisions and exploit transient energy regions. H-CMARL-DE maintains similar path-length efficiency to CTDE (within 1 % shorter path), while improving energy gain and robustness.

Runtime efficiency: Table 4 summarizes execution times for different methods across increasing grid sizes. Heuristic methods (CFA* and CFA*EH) require substantial precomputation and scale poorly with environment size. In contrast, both CTDE and H-CMARL-DE operate in real time. Notably, H-CMARL-DE achieves consistently lower planning latency than CTDE—approximately 34% faster on average—due to its hierarchical structure, which reduces per-step decision complexity.

Statistical validation (ANOVA): To determine whether performance differences across planning methods are statistically significant, a one-way Analysis of Variance (ANOVA) was conducted for each grid size

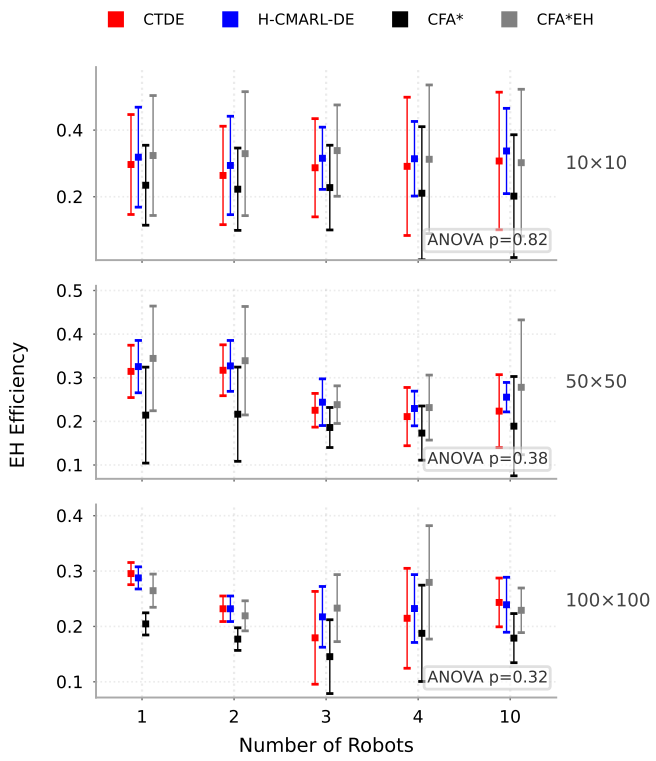


Figure 3: Energy-harvesting efficiency comparison. H-CMARL-DE matches or exceeds CFA*EH despite dynamic operation, and improves EH efficiency over CTDE by 7.3 % on average, demonstrating the benefits of its hierarchical control strategy.

Table 4: Comparison of heuristic and dynamic methods in terms of adaptivity and average runtime.

Method	**Adaptivity**	**10×10**	**50×50**	**100×100**
CFA*	Static, precomputed	0.9 s	8.1 s	15.2 s
CFA*EH	Static, preweighted	1.6 s	11.3 s	17.4 s
CTDE	Real-time, adaptive	44 ms	428 ms	4.2 s
H-CMARL-DE	Real-time, adaptive	29 ms	302 ms	3.4 s

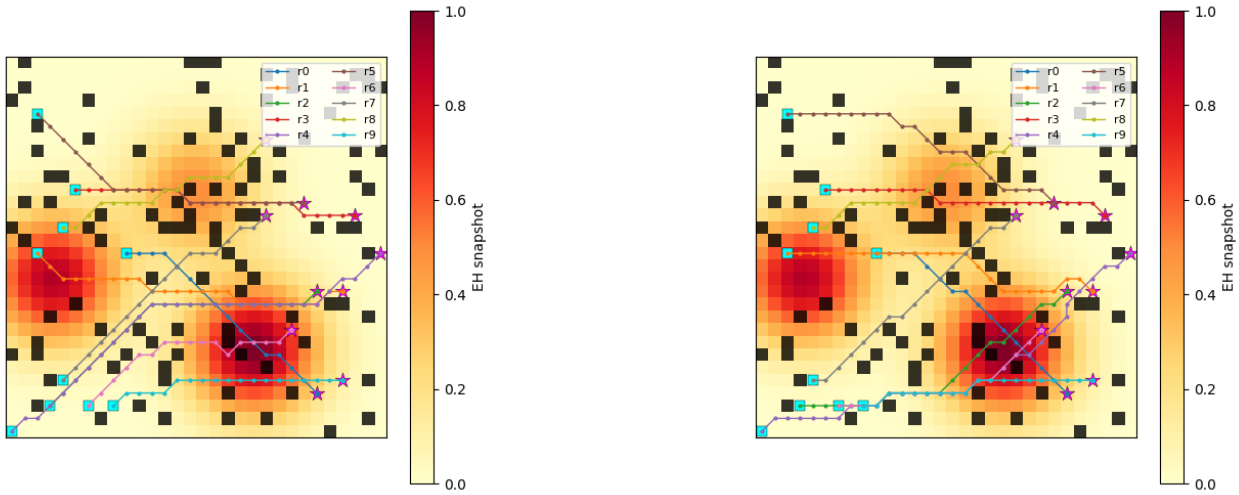
on two key metrics: path-length efficiency and energy-harvesting efficiency. The test evaluates whether the mean performance of all methods is equal or if at least one method differs significantly. Formally, the hypotheses are defined as:

$$H_0 : \mu_{\text{CFA*}} = \mu_{\text{CFA*EH}} = \mu_{\text{CTDE}} = \mu_{\text{H-CMARL-DE}}, \qquad H_1 : \exists\, i, j \text{ such that } \mu_i \neq \mu_j,$$

where μ_i denotes the population mean of the evaluated metric (path-length or energy-harvesting efficiency) for method i, averaged over ten independent trials per grid configuration. Across all grid sizes (10×10, 50×50, and 100×100), the ANOVA test yielded $p_{\text{pl}} = 0.0168$ for path-length efficiency and $p_{\text{eh}} = 0.8236$ for energy-harvesting efficiency. Since $p_{\text{pl}} < 0.05$, we reject the null hypothesis for path-length efficiency, indicating that at least one method, specifically H-CMARL-DE, exhibits a statistically significant difference in trajectory optimization compared to the others. Conversely, $p_{\text{eh}} > 0.05$ implies no statistically significant difference in mean energy-harvesting efficiency across methods.

Summary: CFA* provides a static baseline for shortest-path planning and CFA*EH increases static energy yield. However, CTDE and particularly H-CMARL-DE provide real-time adaptability with significantly lower per-step planning time. H-CMARL-DE's hierarchical structure yields faster execution, 7 % better energy efficiency, and comparable path performance, making it the preferred approach for energy-aware multi-robot navigation in spatiotemporally varying environments.

Figure 4 show one example for planned paths generated using our method for energy-harvesting (EH) aware and non-EH aware scenarios, respectively. The EH-aware path demonstrates how the system optimizes the trajectory to utilize available energy resources efficiently, in comparison to the non-EH-aware path.



(a) EH-aware planned path ($\eta = 3$).

(b) Non-EH-aware planned path ($\eta = 0$).

Figure 4: Planned paths generated using our H-CMARL-DE method for energy-harvesting (EH) aware and non-EH aware scenarios.

5.2.2 EC2: Effectiveness of RL Techniques

This section compare the training session results for H-CMARL-DE method using two different RL algorithm of PPO and A2C. Based on our experiments, PPO consistently outperforms A2C in high-dimensional path-planning tasks, despite comparable performance in simpler environments. PPO's clipped surrogate objective restricts large policy updates, enhancing learning stability and sample efficiency [37]. This mechanism helps prevent destabilizing gradient steps common in complex environments.

In contrast, A2C [38] benefits from efficient parallelism but lacks PPO's clipping, resulting in more aggressive policy updates and less stable learning in high-dimensional spaces. Empirical results in continuous-control benchmarks show PPO provides smoother learning curves and faster convergence compared to A2C [39].

Here, the effectiveness of PPO and A2C is evaluated based on their convergence times and step times across varying grid sizes, robot numbers, and obstacle numbers. The results in Table 5 highlight significant differences in the learning time and step time of these methods under varying levels of environmental complexity.

Convergence time is the duration needed for a learning algorithm to reach a stable state where performance no longer changes significantly and the gained reward is stable. Step time is the average time taken

for a single step in the learning process.

Effect of Grid Size: When the grid size changed from 10×10 to 20×20, the convergence time for PPO rose by 312.1%, while A2C's convergence time increased by 329.4%. This indicates that as the environment's spatial complexity grows, both methods require significantly more converging time. However, PPO shows better performance in managing larger state spaces. A2C, in contrast, struggles more with scalability, as its reliance on synchronous updates introduces higher computational overhead in larger environments.

Effect of Number of Robots: Varying the number of robots highlighted PPO's robustness in multi-agent scenarios. When the robot count increased from 1 to 10, PPO's convergence time rose by 289.3%, while A2C's convergence time escalated by 507.6%. As the number of robots grows, increased interactions make coordination and goal-reaching more difficult. The longer convergence time for 10 robots does not indicate poor scalability. In the 10×10 grid, high density and the presence of static and dynamic obstacles make collision-free pathfinding harder, naturally increasing convergence time. Larger or less congested environments do not show this effect, confirming that the method scales well. As an illustration, Figure 5 shows a PPO learning performance while changing the number of existing robots in the environment.

Effect of Obstacle Number: Increasing the number of obstacles from 10 to 20 further emphasized PPO's adaptability. For PPO, the convergence time increased by 176.5%, while A2C's time surged by 240.1%. Obstacles add complexity to the environment by creating additional challenges in navigation, making it harder for agents to reach their goals.

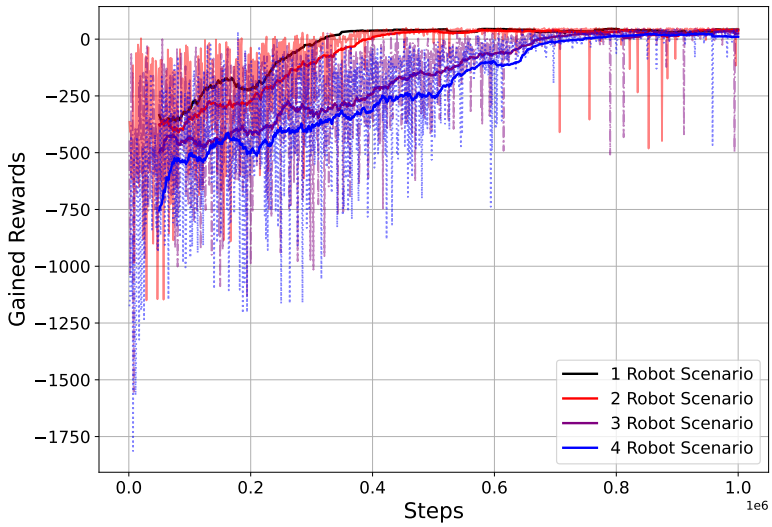


Figure 5: Training Steps for PPO for four cases of 1 robot, 2 robots, 3 robots, and 4 robots scenario. The lines show the average trend of collecting rewards. The area around the lines shows the rewards in each step.

Results show that greater environmental complexity (larger grids, more robots or obstacles) increases convergence time, highlighting a limitation of RL in path planning. PPO scaled more efficiently than A2C across all scenarios.

5.3 Ablation Study

We conducted an ablation study to investigate the effect of reward weighting on policy behavior, focusing on the balance between goal achievement and energy harvesting. Specifically, we varied the weighting coefficient η in Equation 18 across $\{0, 1, 2, 3, 5, 10\}$ and evaluated two performance metrics: PL efficiency and

Table 5: Training and Convergence Times (with standard deviations) for EH-aware Path Planning using PPO and A2C. Conv. is convergence time (seconds), Obs. is the number of obstacles.

Case	**Method**	**Grid**	**Obs.**	**Robots**	**Step time (μs)**	**Conv. (s)**
1	PPO	10×10	10	2	625 ± 110	420 ± 24
2	A2C	10×10	10	2	564 ± 94	1260 ± 78
3	PPO	20×20	10	2	1543 ± 176	1311 ± 96
4	A2C	20×20	10	2	2100 ± 216	4150 ± 185
5	A2C	10×10	10	1	456 ± 38	720 ± 46
6	PPO	10×10	10	1	495 ± 42	394 ± 26
7	A2C	10×10	10	4	816 ± 95	1840 ± 110
8	PPO	10×10	10	4	897 ± 96	795 ± 56
9	A2C	10×10	10	10	1968 ± 215	3655 ± 159
10	PPO	10×10	10	10	980 ± 185	1140 ± 96
11	PPO	10×10	10	3	805 ± 115	765 ± 56
12	A2C	10×10	10	3	815 ± 84	1670 ± 142
13	PPO	10×10	15	3	858 ± 115	895 ± 87
14	A2C	10×10	15	3	1100 ± 136	2460 ± 160
15	PPO	10×10	20	3	923 ± 112	1350 ± 69
16	A2C	10×10	20	3	2100 ± 201	4010 ± 211

EH efficiency. The experiments were carried out using both PPO and A2C for the H-CMARL-DE method. The results are summarized in Figure 6.

For low to moderate values of η (1–3), PL efficiency remains close to baseline ($\eta = 0$), while EH efficiency increases moderately. This indicates that introducing a small energy-harvesting term improves energy collection without substantially affecting goal-directed navigation. Training stability in this regime is also reflected in the relatively low standard deviations across repeated runs.

As η increases to higher values (5 and 10), both PL and EH metrics rise sharply, and the variance across seeds increases. This trend suggests that when the energy reward is heavily weighted, the learned policies adjust their trajectories to exploit harvesting opportunities, sometimes at the cost of shorter and more direct paths. This behavior is consistent across PPO and A2C, although differences in magnitude and variability indicate some algorithm-dependent sensitivity to reward scaling.

Overall, the results highlight the importance of appropriately tuning η to balance competing objectives. Small to moderate η values preserve efficient goal-oriented behavior while enhancing energy collection, whereas excessively large η values bias learning toward energy harvesting and introduce greater variability in policy outcomes. Based on this analysis, we selected $\eta = 3$ as a balanced setting for the main experiments.

Figure 7 shows the impact of robot step size on performance metrics. It is shown that larger step sizes improve energy harvesting efficiency, while shorter steps tend to reduce path length efficiency and success rate to 90% for step size 0.25 and 95% for step size 0.5 in comparison to 99% for step size 1. The reduction in success rate caused by a smaller step size can be attributed to the increased complexity of the task, which requires a more sophisticated policy and makes the learning process more difficult.

It's important to note that these findings are based on a fixed number of training steps (2 million steps); longer training may further clarify these trends.

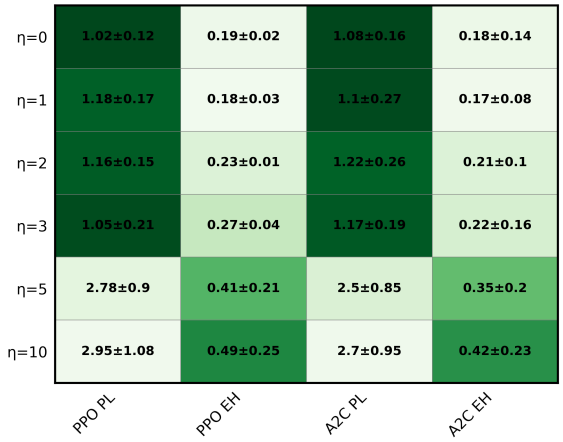


Figure 6: Ablation study of reward weighting using different values of the energy-harvesting coefficient η. Rows correspond to increasing η values, and columns compare PPO and A2C under performance-only (PL) and energy-augmented (EH) reward settings for H-CMARL-DE algorithm. Darker green indicates better performance. For small to moderate η values, path-length efficiency remains stable while energy-harvesting performance improves moderately. At larger η values, both metrics increase but with higher variance, reflecting a shift toward energy-focused behavior. These results illustrate the trade-off between goal achievement and energy collection induced by reward weighting.

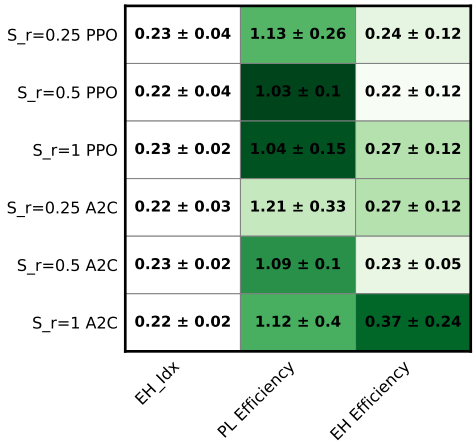


Figure 7: Ablation study of robot step size on path length efficiency, energy harvesting efficiency. Larger step sizes lead to improved energy harvesting, while smaller step sizes (e.g., 0.25 and 0.5) reduce path length efficiency. The darker green blocks show better performance.

6 Case Study and RL Methods Implementation

In this section, two specific maps and harvesting methods were considered to validate the model using ROS and Gazebo simulation. Layout 1, shown in Figure 8, features a warehouse map equipped with wireless induction charging paths that enable the robot to harvest energy while moving on these paths. Also, Layout 2 in this figure presents a scenario where robots can harvest from artificial light sources using mounted solar panels.

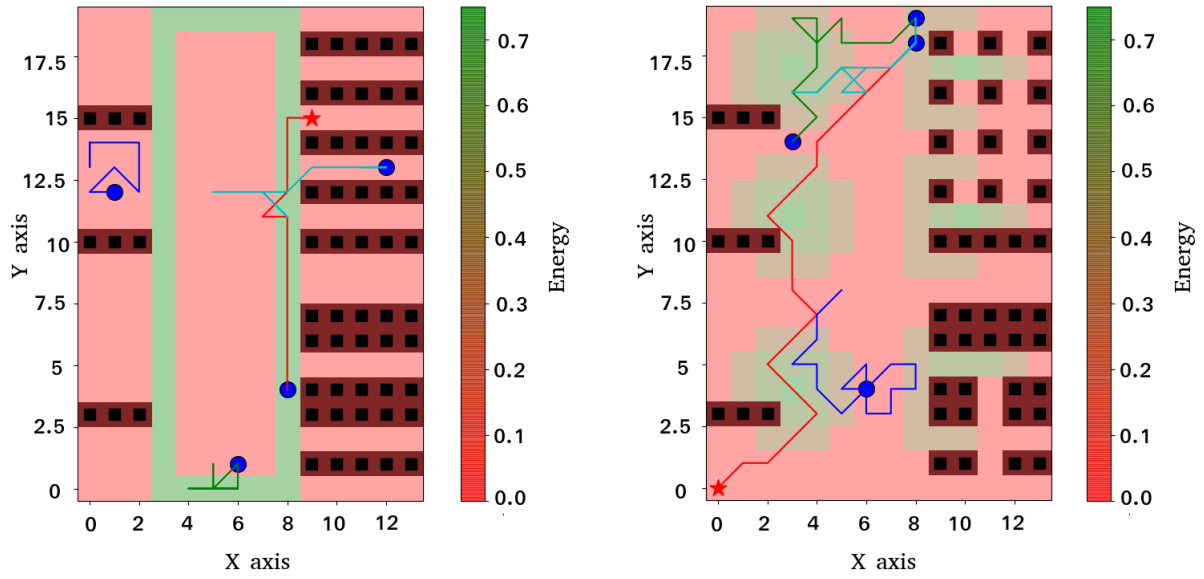


Figure 8: This map shows Layout 1 (left), which is equipped with a rectangular induction charging path model, and Layout 2 (right), which is equipped with a 6-bulb illumination model. The red path and the red star show the path and goal of the robot engaged in path planning and the other paths show the other robot's route.

6.1 ROS-Gazebo Set up

To validate the novel energy-harvesting-aware path planning method, ROS and Gazebo are used for real-time visualization and analysis. ROS uses the Open Motion Planning Library (OMPL Planner or MoveIt), which uses the Rapidly-exploring Random Tree RRT and Probabilistic Roadmap (PRM) Algorithms for path-planning as a default option [40]. In this study, the TurtleBot3 Burger—a compact robot equipped with sensors and features designed for path-planning and energy-harvesting simulations—is utilized. Figure 9 shows the RViz and Gazebo environment and the map used for this analysis.

Our system is built around a structured observe, think, act cycle that enables responsive decision-making in dynamic environments. Figure 10 illustrates the operational loop followed by the RL agent at each decision step.

- **Map Input**: The agent receives a map that includes harvesting grid locations, static obstacles, and the predicted paths of other robots. This map can be created either manually or automatically using imaging technologies such as satellite imagery, aerial drone footage, or LiDAR scans. Image processing techniques, including segmentation, object detection, and classification, are used to extract relevant features: obstacles are identified based on shape and texture; energy harvesting areas (e.g., solar-rich zones or charging hotspots) are inferred from infrared or irradiance data; and energy consumption zones (e.g., rough terrain or elevation changes) are derived from elevation models or material classification using multispectral imaging. During deployment, the map is further updated in real time based on the robot's onboard sensors, such as LiDAR, RGB-D cameras, inertial measurement units (IMUs), and power usage sensors. This allows the system to reflect dynamic changes in the environment, including new obstacles, terrain variation, or altered energy availability. The resulting map is represented as a discretized grid G, where each cell contains metadata relevant for obstacle avoidance, energy-aware planning, and multi-agent coordination.

- **Observe**: At each step, the robot captures its current observation of the environment, which includes both local context and immediate surroundings. This observation is acquired using onboard sensors

such as LiDAR, RGB or RGB-D cameras, ultrasonic range finders, and IMUs (Inertial Measurement Units). Accurate sensing requires synchronization between sensor data streams and the robot's movement to ensure that observations correspond precisely to the robot's actual position and orientation at each time step. This synchronization is critical for reliable perception, mapping, and decision-making in dynamic or partially observable environments.

- **Think (RL Agent Decision)**: The RL agent processes this observation and selects the next optimal action based on the learned policy.

- **Act (Action Execution)**: The robot executes the selected action, typically a movement to a neighboring grid cell based on the predefined grid structure and step size. Ensuring accurate movement requires synchronization between the planned steps and the robot's physical motion. This synchronization can be achieved through calibration or iterative trial-and-error to determine an appropriate and reliable step size that aligns with the grid resolution. In our ROS-Gazebo implementation we synchronized the step size movements using trial and error.

- **Goal Check**: If the robot reaches its assigned goal, the task terminates; otherwise, the cycle repeats.

While observations may sometimes appear unchanged between steps—potentially leading to the interpretation of an open-loop system—our framework reflects a *quasi-dynamic* environment. Specifically, during each path planning cycle, some parameters (such as static obstacles and pre-planned robot paths) are held constant. These are updated in subsequent cycles to account for environmental changes. This simplification is a practical trade-off based on current learning constraints and is appropriate for structured settings.

Importantly, our system is fundamentally closed-loop, as the RL agent continuously reacts to changes in the environment. The model is trained to inherently respond to dynamic elements like moving obstacles, removing the need for a separate collision-avoidance module.

In real-world operation, the RL agent executes the observe, think, act loop in real time. At every time step, it receives an updated observation and selects the next move accordingly. The final policy thus represents a sequence of adaptive, environment-aware decisions.

Two factors are critical for practical deployment:

- **Decision Scheduling**: The system maintains continuous perception and decision-making at each time step, allowing dynamic responsiveness. This synchronization should align with the robot's actual movement speed.

- **Computation Speed**: The trained model's inference time per grid step is on the order of microseconds (tested on an Intel i7 12th-generation processor), ensuring rapid response and real-time feasibility.

While real-world challenges such as coordination, communication, and synchronization remain, the proposed approach demonstrates strong potential for application in industrial and logistics environments where real-time operation is essential.

6.2 Simulation Results

The results summarized in Table 6 demonstrate that all configurations achieved a 100% success rate in multi-robot navigation tasks across ten independent trials, confirming robustness and stable convergence of both learning-based and classical methods.

In the 2-robot setup, the baseline ROS-OMPL planner achieved the highest PL efficiency (1.11), reflecting more direct paths. However, the proposed H-CMARL-DE framework achieved a substantially higher EH efficiency (0.25 vs. 0.12), along with the highest final reward (68.65), indicating better long-term energy–task trade-offs. CTDE achieved slightly lower EH efficiency (0.21) and reward (64.65), consistent with its reliance

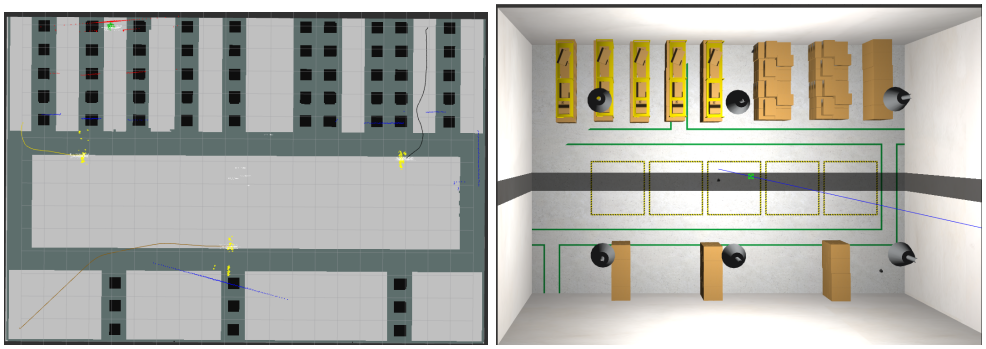


Figure 9: Example of a navigation path using ROS-RViz (left) and Gazebo simulator (right) in the use-case map environment. The yellow spots are the robots and the lines show their corresponding planned path.

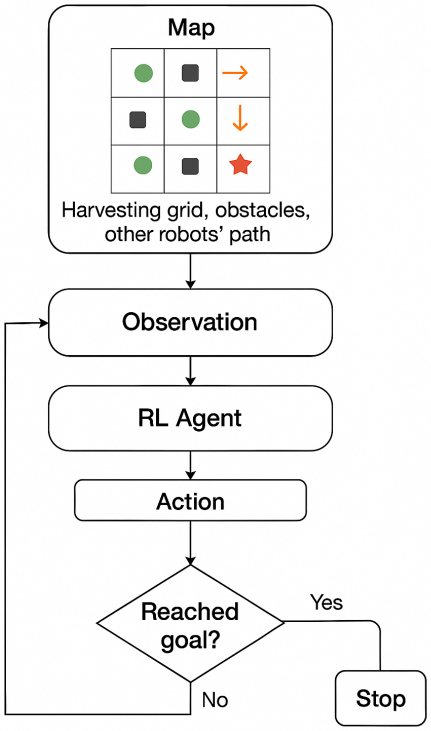


Figure 10: Flowchart illustrating the observe–think–act cycle in a reinforcement learning-based robot navigation system.

on centralized critics that limit adaptive reweighting during decentralized execution. Although H-CMARL-DE incurred a modestly longer mission completion time (MCT = 56.2 s) compared to ROS-OMPL (51.5 s), the added duration corresponds to active energy-harvesting maneuvers.

In the 4-robot configuration, similar trends were observed. ROS-OMPL maintained the shortest paths (PL eff. 1.08) but lacked collision awareness and energy optimization, while H-CMARL-DE achieved the best EH efficiency (0.29) and the highest reward (61.65). CTDE followed closely with 0.24 EH efficiency and 58.65 reward. Despite slightly higher MCT values, the learned policies demonstrated better scalability, adaptability, and coordination under dynamic energy and obstacle conditions.

Overall, these results confirm that H-CMARL-DE improves EH efficiency by up to 240% over classical ROS-based planning, while maintaining comparable path efficiency and manageable computation time. The

meta-level Differential Evolution optimization enables adaptive tuning of hierarchical policies and priorities, yielding both energy-aware and collision-free trajectories in complex multi-agent settings.

Table 6: Performance comparison of ROS-OMPL, CTDE, and H-CMARL-DE in multi-robot energy-aware navigation. Metrics include final goal-based reward, energy-harvesting efficiency (AEH/APL), path-length efficiency (APL/ACPL), obstacle count, and MCT. H-CMARL-DE achieves the highest energy efficiency and cumulative reward while maintaining feasible path quality and timing.

Method	**Robot**	**Layout**	**Average Reward**	**EH Eff.**	**PL Eff.**	**Obstacles**	**MCT (s)**
ROS-OMPL	2	1	-	0.12 ± 0.03	1.110 ± 0.09	59	51.50 ± 5.8
H-CMARL-DE	2	1	68.65 ± 6.5	0.25 ± 0.05	1.17 ± 0.12	59	56.20 ± 5.7
CTDE	2	1	64.65 ± 7.1	0.21 ± 0.07	1.19 ± 0.14	59	58.50 ± 7.3
ROS-OMPL	4	2	-	0.13 ± 0.02	1.08 ± 0.09	48	47.12 ± 4.1
H-CMARL-DE	4	2	61.65 ± 5.4	0.29 ± 0.09	1.13 ± 0.08	48	51.10 ± 5.3
CTDE	4	2	58.65 ± 6.5	0.24 ± 0.08	1.17 ± 0.12	48	54.70 ± 5.6

7 Conclusion

This paper presented Hierarchically Conditioned Multi-Agent RL with Meta-DE (H-CMARL-DE), a framework for multi-robot energy-aware path planning. The method integrates reinforcement learning with evolutionary meta-optimization, allowing robots to coordinate efficiently while maintaining decentralized execution. The hierarchical design ensures structured time–space coordination without violating decentralized policy learning principles.

H-CMARL-DE consistently outperforms ROS-OMPL and CTDE, achieving up to 240 % higher energy-harvesting efficiency while keeping path-length efficiency within 10 % of the classical planner. Meta-level DE enables adaptive tuning, yielding the highest cumulative rewards across all layouts, while requiring less computation than CTDE.

Compared with CTDE, which relies on joint gradient updates and global critics, H-CMARL-DE reduces non-stationarity by enforcing a lower-triangular dependency chain among agents. This improves scalability and stability in multi-agent settings, where communication and synchronization overhead are critical bottlenecks. The hierarchical reservation mechanism further ensures strict safety by preventing both vertex and edge collisions during decentralized execution.

Experiments conducted in simulated ROS environments demonstrated the robustness and adaptability of H-CMARL-DE under dynamic obstacle and energy-field variations. Although a slight increase in mission completion time was observed, this trade-off corresponds to intentional energy-harvesting maneuvers, not inefficiency.

Future work will explore continuous action spaces, real-world hardware validation, and the integration of online DE adaptation for real-time parameter tuning. The proposed framework establishes a foundation for scalable, collision-free, and energy-efficient cooperative robot navigation in dynamic and resource-aware environments.

Acknowledgment

This project has received funding from the European Union's Horizon Europe Framework Program under Grant Agreement No. 101093046. Views and opinions expressed are however those of the author(s) only and the European Commission is not responsible for any use that may be made of the information it contains. Moreover, part of the research presented in this paper is funded by the Flemish Government under FWO Project G019722N - LOCUSTS.

Conflict of Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] M. Mokhtari, P. H. A. Mohamadi, M. Aernouts, R. K. Singh, B. Vanderborght, M. Weyn, and J. Famaey, "Energy-aware multi-robot task scheduling using meta-heuristic optimization methods for ambiently-powered robot swarms," *Robotics and Autonomous Systems*, vol. 186, p. 104898, 2025.

[2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction.* MIT Press, 2018.

[3] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[4] P. Zhao, Y. Chang, W. Wu, H. Luo, Z. Zhou, Y. Qiao, Y. Li, C. Zhao, Z. Huang, B. Liu, X. Liu, S. He, and D. Guo, "Dynamic rrt: Fast feasible path planning in randomly distributed obstacle environments," *Journal of Intelligent & Robotic Systems*, vol. 107, no. 4, p. 48, 2023.

[5] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[6] X. Liu, S. Wen, Y. Hu, F. Han, H. Zhang, and H. R. Karimi, "An active slam with multi-sensor fusion for snake robots based on deep reinforcement learning," *Mechatronics*, vol. 103, p. 103248, 2024.

[7] S. Wen, Z. Wen, D. Zhang, H. Zhang, and T. Wang, "A multi-robot path-planning algorithm for autonomous navigation using meta-reinforcement learning based on transfer learning," *Applied Soft Computing*, vol. 110, p. 107605, 2021.

[8] X. Li, H. Zhang, and Y. Chen, "Bidirectional obstacle avoidance enhancement–deep deterministic policy gradient for autonomous navigation," *Advanced Intelligent Systems*, 2024.

[9] J. Xue, J. Chen, and S. Zhang, "Action-curiosity-based deep reinforcement learning algorithm for path planning in a nondeterministic environment," *Intelligent Computing*, vol. 4, 2025.

[10] J. Xue, M. He, J. Chen, and B. Dong, "Improved ddpg based on enhancing decision evaluation for path planning in high-density environments," *Expert Systems with Applications*, vol. 279, p. 127378, 2025.

[11] H. Zhang, Y. Zhang, C. Liu, and Z. Zhang, "Energy efficient path planning for autonomous ground vehicles with ackermann steering," *Robotics and Autonomous Systems*, vol. 162, p. 104364, 2023.

[12] S. Doğru and L. Marques, "Eco-cpp: Energy constrained online coverage path planning," *Robotics and Autonomous Systems*, vol. 157, p. 104242, 2022.

[13] G. Aiello, K. P. Valavanis, and A. Rizzo, "Fixed-wing uav energy efficient 3d path planning in cluttered environments," *Journal of Intelligent & Robotic Systems*, vol. 105, p. 60, 2022.

[14] C. Wei, Y. Li, Y. Ouyang, and Z. Ji, "Deep reinforcement learning with heuristic corrections for ugv navigation," *Journal of Intelligent & Robotic Systems*, vol. 109, p. 18, 2023.

[15] C. Tao, M. Li, F. Cao, Z. Gao, and Z. Zhang, "A multiobjective collaborative deep reinforcement learning algorithm for jumping optimization of bipedal robot," *Advanced Intelligent Systems*, vol. 6, no. 1, 2023.

[16] M. Visca, R. Powell, Y. Gao, and S. Fallah, "Deep meta-learning energy-aware path planner for unmanned ground vehicles in unknown terrains," *IEEE Access*, vol. 10, pp. 30 055–30 068, 2022.

[17] O. Bouhamed, X. Wan, H. Ghazzai, and Y. Massoud, "A ddpg-based approach for energy-aware uav navigation in obstacle-constrained environment," in *2020 IEEE 6th World Forum on Internet of Things (WF-IoT)*, 2020, pp. 1–6.

[18] A. Niaraki, J. Roghair, and A. Jannesari, "Visual exploration and energy-aware path planning via reinforcement learning," *arXiv: 1909.12217*, 2021.

[19] H. Huang, A. V. Savkin, and C. Huang, "Energy-efficient path planning of solar-powered uavs for communicating with mobile ground users in urban environments," in *Wireless Communication Networks Supported by Autonomous UAVs and Mobile Ground Robots.* Elsevier, 2022, ch. 15.

[20] H. Huang and A. V. Savkin, "Energy-efficient decentralized navigation of a team of solar-powered uavs for collaborative eavesdropping on a mobile ground target in urban environments," *Ad Hoc Networks*, vol. 117, p. 102485, 2021.

[21] W. Ni, Y. Bi, D. Wu, and X. Ma, "Energy-optimal trajectory planning for solar-powered aircraft using soft actor-critic," *Chinese Journal of Aeronautics*, vol. 35, no. 6, 2021.

[22] Y. Liao, K. Li, Y. Zhao, H. Tang, M. Zhang, X. Liu, and Z.-M. Yuan, "Energy optimal path planning of ocean-energy driven unmanned surface vehicles," *Ocean Engineering*, vol. 327, p. 120965, 2025.

[23] Z. Hu and S. Liu, "Research on route planning for solar uav based on the intelligent optimization algorithm," *Proceedings of the Institution of Mechanical Engineers, Part G: Journal of Aerospace Engineering*, 2023.

[24] A. Hasankhani, E. B. Ondes, Y. Tang, C. Sultan, and J. H. van Zwieten, "Integrated path planning and tracking control of marine current turbine in uncertain ocean environments," in *2022 American Control Conference (ACC).* Atlanta, GA, USA: IEEE, 2022, pp. 3106–3113.

[25] S. Li, "Advanced algorithms for autonomous guidance of solar-powered uavs," *arXiv preprint arXiv:2404.02920*, 2024.

[26] W. Ni, Y. Bi, D. Wu, and X. Ma, "Energy-optimal trajectory planning for solar-powered aircraft using soft actor-critic," *Chinese Journal of Aeronautics*, vol. 35, no. 10, pp. 337–353, 2022.

[27] P. Plonski and P. Tokekar, "Energy-aware coverage path planning with solar-powered robots," *Journal of Field Robotics*, vol. 30, no. 5, pp. 699–717, 2013.

[28] S. Das and P. N. Suganthan, "Differential evolution: A survey of the state-of-the-art," *IEEE Transactions on Evolutionary Computation*, vol. 15, no. 1, pp. 4–31, 2011.

[29] B. Hengst, *Hierarchical Reinforcement Learning.* Boston, MA: Springer US, 2017, pp. 611–619.

[30] G. Drappo, A. M. Metelli, and M. Restelli, "A provably efficient option-based algorithm for both high-level and low-level learning," *arXiv preprint arXiv:2406.15124*, 2024.

[31] P. Hernandez-Leal, M. Kaisers, T. Baarslag, and E. Muñoz de Cote, "A survey of learning in multiagent environments: Dealing with non-stationarity," *arXiv preprint arXiv:1707.09183*, 2017.

[32] G. Papoudakis, F. Christianos, A. Rahman, and S. V. Albrecht, "Dealing with non-stationarity in multi-agent deep reinforcement learning," *Journal of Machine Learning Research*, vol. 22, no. 230, pp. 1–75, 2021.

[33] H.-T. L. Chiang, A. Faust, M. Fiser, and A. Francis, "Learning navigation behaviors end-to-end with autorl," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 2007–2014, 2019.

[34] N. Awad, N. Mallik, and F. Hutter, "Dehb: Evolutionary hyperband for scalable, robust and efficient hyperparameter optimization," *arXiv preprint arXiv:2105.09821*, 2021.

[35] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," *arXiv preprint arXiv:1606.01540*, 2016.

[36] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[37] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[38] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," *arXiv preprint arXiv:1602.01783*, 2016.

[39] Y. Huang *et al.*, "A comparative study of deep reinforcement learning models: Dqn vs ppo vs a2c," *arXiv preprint arXiv:2407.14151*, 2024.

[40] ROS Planning and MoveIt! Community, *OMPL Planner — MoveIt! Tutorials Melodic Documentation*, ROS.org, 2021.

Mohmmadsadegh Mokhtari received a Master's degree in Electrical Engineering from Shiraz University of Technology. He is currently pursuing a Ph.D. in Computer Science at the University of Antwerp. Along with his studies, he is researching energy management in the fields of robotics and computer science at IDLab, IMEC.

Bram Vanderborght was born in Belgium in 1980. He received his degree in Mechanical Engineering from the Vrije Universiteit Brussel in 2003 with the highest distinction. Since 2003, he has been a researcher at the VUB, supported by the Fund for Scientific Research Flanders (FWO).

Jeroen Famaey is an associate research professor at the Department of Computer Science of the University of Antwerp and IMEC, Belgium. His research focuses on performance modeling and optimization of wireless network protocols. He has published over 180 peer-reviewed journal articles and conference papers, and 8 granted patents. He was listed as one of the top 2% most cited scientists worldwide by Stanford University both in 2022 and 2023.